

اونيۏرسيتي مليسيا ڤهڠ السلطان عبد الله
UNIVERSITI MALAYSIA PAHANG
AL-SULTAN ABDULLAH

BSD2343 DATA WAREHOUSING
SEMESTER 2 2023/2024

TITLE :
DESIGN AND IMPLEMENTATION OF A DATA WAREHOUSE FOR INSURANCE CHURN ANALYSIS
(SDG 8: Decent Work and Economic Growth)

PREPARED FOR :
DR AZUANA BINTI RAMLI

NAME	STUDENT ID	SECTION
ZAHIN ZIKRI BIN ZAWAWI	SD23019	02G
MUHAMMAD HARITH DANIAL BIN RAZAK	SD23005	02G
MUHAMMAD AQIL BIN MOHAMMAD ASHRAF	SD23052	02G
MOHAMMAD NOR'AZHARI ADHAR BIN NOR'AINA	SD23058	01G
AHMAD AFIQ HILMY BIN HAZA HILMY	SD23009	01G

TABLE OF CONTENT

1.0 BACKGROUND

1.1 Project Background

1.2 Description of Data

1.3 Problem Statement

1.4 Objectives

1.5 Data Schema

2.0 ARCHITECTURE

2.1 Pipeline Structure

2.2 Flowchart

3.0 DATABASE

3.1 Relational Model

3.2 Relational between Data

3.3 Identification of Data Warehouse Schema

4.0 ETL PIPELINE

4.1 ETL Pipeline

4.2 ETL Process

4.2.1 Extract

4.2.2 Transform

4.2.3 Load

5.0 DATA ANALYSIS AND VISUALISATION

6.0 CONCLUSION

7.0 REFERENCES

8.0 APPENDIX

1.0 BACKGROUND

1.1 Project Background

The research project focuses on understanding customer behavior and retention among the insurance industry, in support of the overall aim of promoting sustainable, customer-oriented business practices. High customer attrition is an issue that affects organizations in the modern competitive business environment. The cost of gaining a new customer often surpasses the cost of retaining an existing one. There are a variety of reasons that contribute to customer attrition that have been identified, such as financial status, demographic age groups, marital status, length of engagement, credit standing, and geographical location. Analysis of these factors will assist insurance organizations to improve their service levels and develop customer relationships that lead to long-term sustainability.

The main aim of this research project is to establish meaningful patterns and trends relating to customer behavior. From an analysis of the patterns, the research would establish which demographic groups have the highest likelihood of disengagement, and what underlying characteristics they share. Finally, the project hopes to aid the formulation of evidence-based strategies that insurance organizations can adopt to reduce attrition, maintain their precious customer pool, and thereby support business growth in an improved efficiency.

In conclusion, The findings from this study can serve as a point of reference for insurance companies, marketing executives, and analysts to refine their customer retention and customer loyalty strategies.

1.2 Description of Data

The data we obtained come from the kaggle website, which consists of all the relevant data and covers a wide range of demographics regarding insurance churn. The data set includes 1,536,673 unique addresses and 2,280,321 unique customers, of which 2,112,579 have demographic information. Additionally, 269,259 customers cancelled their policies within the previous year. It consists of 7 separate tables that may or may not relate to each other. 3 fact tables and 4 dimension tables.

Fact Tables

Fact_Customer_Profile Table

No	Variable	Data Type	Description
1	INDIVIDUAL_ID (PK)	Numeric	Unique ID for a specific insurance customer
2	ADDRESS_ID	Numeric	Latitude of the address
3	CURR_ANN_AMT	Numeric	Longitude of the address
4	DAYS_TENURE	String	Mailing Address of the address

Fact_Account_Status Table

No	Variable	Data Type	Description
1	INDIVIDUAL_ID (PK)	Numeric	Unique ID for a specific insurance customer
3	ACCT_SUSPD_DATE	Date	Date of Account Suspension or Cancellation
3	CHURN	Numeric	1 - Customer churned/left the company 0 - Customer does not churned/left the company

Fact_Geographic

No	Variable	Data Type	Description
1	INDIVIDUAL_ID (PK)	Numeric	Unique ID for a specific insurance customer
2	ADDRESS_ID	Numeric	Unique ID for the primary address associated with a customer
3	CHURN	Numeric	1 - Customer churned/left the company 0 - Customer does not churned/left the company

Dimension Tables

Address Table

No	Variable	Data Type	Description
1	ADDRESS_ID	Numeric	Unique ID for a specific address
2	INDIVIDUAL_ID (PK)	Numeric	Unique ID for a specific insurance customer
3	LATITUDE	Numeric	Latitude of the address
4	LONGITUDE	Numeric	Longitude of the address
5	CITY	String	City
6	STATE	String	State
7	COUNTY	String	County

Customer Table

No	Variable	Data Type	Description
1	ADDRESS_ID	Numeric	Unique ID for a specific address
2	INDIVIDUAL_ID (PK)	Numeric	Unique ID for a specific insurance customer

3	CURR_ANN_AMT	Numeric	The Annual dollar value paid by the customer. It is not the policy amount. It is actually the amount the customer paid during the previous year.
4	DAYS_TENURE	Numeric	The time in days individual has been a customer with the insurance agency
5	AGE_IN_YEARS	Numeric	Age of the individual
6	DATE_OF_BIRTH	Date	Individual's date of birth
7	CUST_ORIG_DATE	Date	The data the individual became a customer

Demographic Table

No	Variable	Data Type	Description
1	INDIVIDUAL_ID (PK)	Numeric	Unique ID for a specific insurance customer
2	INCOME	Numeric	Estimated Income for the household associated with the individual
3	HAS_CHILDREN	Numeric	1- Individual has children 0 - otherwise
4	LENGTH_OF_RESIDENCE	String	Estimated number of years the individual has lived in their current home
5	MARITAL_STATUS	String	Marital status
6	HOME_MARKET_VALUE	Numerical	Minimum Value of Home Market
7	HOME_OWNER	Numeric	1- Individual owns primary home 0- otherwise
8	COLLEGE_DEGREE	Numeric	1- individual has college degree 0- otherwise

9	GOOD_CREDIT	Numeric	1- Individual has FICO greater than 630 0 - otherwise

Termination table

No	Variable	Data Type	Description
1	INDIVIDUAL_ID (PK)	Numeric	Unique ID for a specific insurance customer
2	ACCT_SUSPD_DATE	Date	Date of Account Suspension or Cancellation

1.3 Problem Statement

Customer attrition is a serious issue that affects the economic sustainability and growth patterns of organizations. Customer attrition is the result of a variety of factors such as customer income levels, lifestyles, period of involvement, credit profile, geographic factors, and major life milestones such as marriage or property ownership. Determining the determinants of customer departures often becomes a challenging task; however, gaining this information is important for organizations that wish to become and remain competitive and develop better customer relationships.

In order to tackle this, implementing a data analysis-based approach becomes a must. It's critical that one accurately identifies the customer segments with the highest probability of canceling their services and maps out the commonalities among them. This information proves extremely valuable since it gives insurance organizations the power to see early warning signs and take strategic steps to secure their most prized customers.

Here are some of the important questions that we aim to answer in this study:

- What are the key personal, geographic, and financial factors that influence customer churn in the auto insurance sector?
- How can data analysis techniques help uncover hidden patterns and relationships between customer characteristics and churn behavior?
- How can the findings be translated into practical insights that insurance providers can use to improve retention strategies?

1.4 Objectives

This research project embarks to fulfil the following objectives:

- To evaluate the key personal, geographical, and financial attributes that contribute significantly to customer churn within the insurance sector.

- To reveal hidden patterns and relationships between customer characteristics and churn tendencies, enabling the identification and segmentation of high-risk customer groups.

- To transform analytical findings into actionable insights that assist insurance firms in formulating proactive client retention strategies and mitigating churn risk.

1.5 Data Schema

A data schema is a collection of database objects, including tables, views, indexes, and synonyms. There are a variety of ways of arranging schema objects in the schema models designed for data warehousing. The mortality dataset consists of 7 Tables which are customer, termination, address, demographic, account status, customer profile and geographic.

We used 2 libraries in Jupyter to display data schema which are pandas and numpy.
import pandas as pd
import numpy as np

- For Customer

```
[35]: customer.dtypes

[35]: individual_id     float64
      address_id        float64
      curr_ann_amt      float64
      days_tenure       float64
      cust_orig_date     object
      age_in_years      float64
      date_of_birth      object
      dtype: object

[37]: customer.info()

      <class 'pandas.core.frame.DataFrame'>
      RangeIndex: 1416734 entries, 0 to 1416733
      Data columns (total 7 columns):
       #   Column          Non-Null Count    Dtype
      ---  ------          --------------    -----
       0   individual_id   1416734 non-null  float64
       1   address_id      1416734 non-null  float64
       2   curr_ann_amt    1416734 non-null  float64
       3   days_tenure     1416734 non-null  float64
       4   cust_orig_date  1416734 non-null  object
       5   age_in_years    1416734 non-null  float64
       6   date_of_birth   1416734 non-null  object
      dtypes: float64(5), object(2)
      memory usage: 75.7+ MB
```

Figure 1.5.1: Customer tables

Based on Figure 1.5.1 above, the data schema for the mortality consists of 7 columns. The data type for this data frame consists of two data types which are float and string. Only one column has string data type while the rest are float data type.

- For Termination

```
[39]: termination.dtypes

[39]: individual_id      float64
      acct_suspd_date     object
      dtype: object

[41]: termination.info()

      <class 'pandas.core.frame.DataFrame'>
      RangeIndex: 1416734 entries, 0 to 1416733
      Data columns (total 2 columns):
       #   Column           Non-Null Count    Dtype
      ---  ------           --------------    -----
       0   individual_id    1416734 non-null  float64
       1   acct_suspd_date  161128 non-null   object
      dtypes: float64(1), object(1)
      memory usage: 21.6+ MB
```

Figure 1.5.2: Termination tables

Based on Figure 1.5.2 above, the data schema for mental health consists of two columns. The data type for this data frame consists of two data types which are float and string..

- For Address

```
[43]: address.dtypes

[43]: individual_id    float64
      address_id       float64
      latitude         float64
      longitude        float64
      city              object
      state             object
      county            object
      dtype: object

[45]: address.info()

      <class 'pandas.core.frame.DataFrame'>
      RangeIndex: 1416734 entries, 0 to 1416733
      Data columns (total 7 columns):
       #   Column         Non-Null Count    Dtype
      ---  ------         --------------    -----
       0   individual_id  1416734 non-null  float64
       1   address_id     1416734 non-null  float64
       2   latitude       1416734 non-null  float64
       3   longitude      1416734 non-null  float64
       4   city           1416734 non-null  object
       5   state          1416734 non-null  object
       6   county         1416734 non-null  object
      dtypes: float64(4), object(3)
      memory usage: 75.7+ MB
```

Figure 1.5.3: Address Table

Based on Figure 1.5.3 above, the data schema for the address consists of seven columns. The data type for this data frame consists of two data types which are string and float. Three columns have string data type while the rest are float data type.

- For Demographic

```
[47]: demographic.dtypes

[47]: individual_id          float64
      income                 float64
      has_children           float64
      length_of_residence    float64
      marital_status          object
      home_market_value       object
      home_owner             float64
      college_degree         float64
      good_credit            float64
      dtype: object

[49]: demographic.info()

      <class 'pandas.core.frame.DataFrame'>
      RangeIndex: 1416734 entries, 0 to 1416733
      Data columns (total 9 columns):
       #   Column               Non-Null Count    Dtype
      ---  ------               --------------    -----
       0   individual_id        1416734 non-null  float64
       1   income               1416734 non-null  float64
       2   has_children         1416734 non-null  float64
       3   length_of_residence  1416734 non-null  float64
       4   marital_status       1416734 non-null  object
       5   home_market_value    1409511 non-null  object
       6   home_owner           1416734 non-null  float64
       7   college_degree       1416734 non-null  float64
       8   good_credit          1416734 non-null  float64
      dtypes: float64(7), object(2)
      memory usage: 97.3+ MB
```

Figure 1.5.4: Demographic Table

Based on Figure 1.5.4 above, the data schema for the risk factors consists of nine columns. The data type for this data frame consists of two data types which are string and float. Only two columns have string data type while the rest are float data type.

- For Account Status

```
[51]: account.dtypes

[51]: individual_id    float64
      churn            float64
      dtype: object

[53]: account.info()

      <class 'pandas.core.frame.DataFrame'>
      RangeIndex: 1416734 entries, 0 to 1416733
      Data columns (total 2 columns):
       #   Column         Non-Null Count    Dtype
      ---  ------         --------------    -----
       0   individual_id  1416734 non-null  float64
       1   churn          1416734 non-null  float64
      dtypes: float64(2)
      memory usage: 21.6 MB
```

Figure 1.5.5: Account Status Table

Based on Figure 1.5.4 above, the data schema for the risk factors consists of two columns. The data type for this data frame consists of one data type which is float.

- For Customer Profile

```
[55]: profile.dtypes

[55]: individual_id    float64
      address_id       float64
      curr_ann_amt     float64
      days_tenure      float64
      dtype: object

[57]: profile.info()

      <class 'pandas.core.frame.DataFrame'>
      RangeIndex: 1416734 entries, 0 to 1416733
      Data columns (total 4 columns):
       #   Column         Non-Null Count    Dtype
      ---  ------         --------------    -----
       0   individual_id  1416734 non-null  float64
       1   address_id     1416734 non-null  float64
       2   curr_ann_amt   1416734 non-null  float64
       3   days_tenure    1416734 non-null  float64
      dtypes: float64(4)
      memory usage: 43.2 MB
```

Figure 1.5.6: Customer Profile Table

Based on Figure 1.5.4 above, the data schema for the risk factors consists of four columns. The data type for this data frame consists of one data type which is float.

- For Geographic

```
[59]: geographic.dtypes

[59]: individual_id    float64
      address_id       float64
      churn            float64
      dtype: object

[61]: geographic.info()

      <class 'pandas.core.frame.DataFrame'>
      RangeIndex: 1416734 entries, 0 to 1416733
      Data columns (total 3 columns):
       #   Column         Non-Null Count    Dtype
      ---  ------         --------------    -----
       0   individual_id  1416734 non-null  float64
       1   address_id     1416734 non-null  float64
       2   churn          1416734 non-null  float64
      dtypes: float64(3)
      memory usage: 32.4 MB
```

Figure 1.5.7: Geographic Table

Based on Figure 1.5.7 above, the data schema for the risk factors consists of three columns. The data type for this data frame consists of one data type which is float.

2.0 ARCHITECTURE

2.1 Pipeline Structure

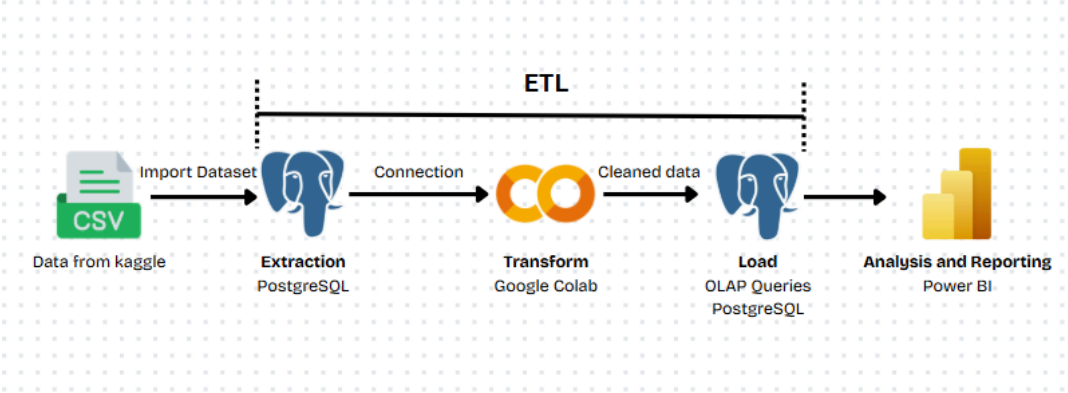


Figure 2.1 The Data Architecture Pipeline

Based on the figure 2.1 above, the insurance churn dataset was obtained from *kaggle*. Initially, the dataset consists of 5 tables. The tables are autoinsurance_churn, customer, demographic, termination and address. A pre-extraction process needs to be done on Microsoft Excel so that data can be extracted into PostgreSQL without errors. In the extraction process, a total of 7 tables were imported, the initial fact table (autoinsurance_churn) was split into 3 separate fact tables namely fact_customer_profile, fact_geographic, fact_account_status.

Next, the transformation process, which is mainly cleaning the data in Google Colab. Before cleaning, data must be connected into Google Collab file. The cleaning process includes replacing missing data, standardising field names, and converting certain data into a more usable format.

After the data is cleaned and ready to use, we will load it back into PostgreSQL for the Online Analytical Process (OLAP). The OLAP operations that will be run are dicing, drill-down, roll-up, slicing, pivot.

Lastly, the data will be imported into Power BI for visualisation. The data visualisation will be used for analysis and final reporting to reach the objectives of this project.

2.2 Flowchart

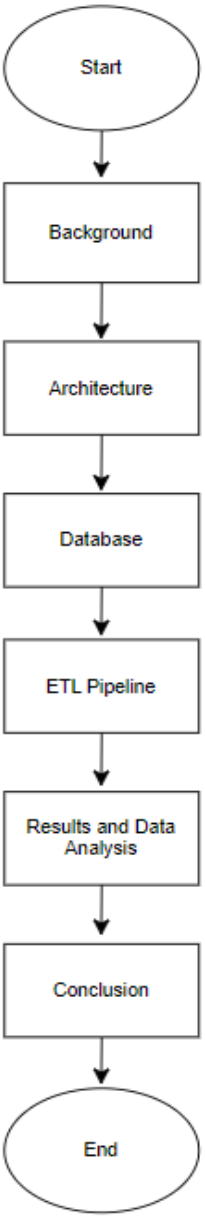


Figure 2.2 Flow of overall of the project

Figure 2.2 shows the flow of the project which includes the background related to data, architecture, database, ETL Pipeline, results and data analysis and also conclusion.

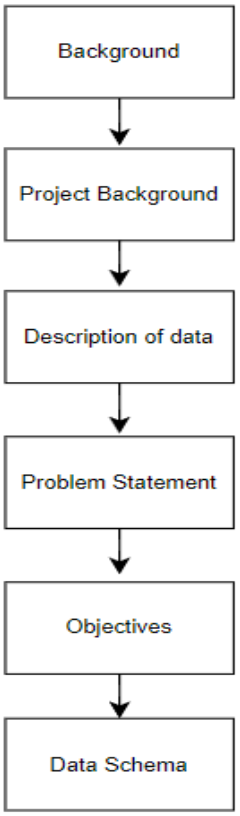


Figure 2.3 Flow of process of Background

Figure 2.3 shows the flow of the background. In this flowchart, it includes the part of project background, description of data, problem statement, objectives, and data schema. The dataset is about auto insurance churn, which will be used to uncover the factors influencing customer churn and the relationships between customer attributes and churn behavior. The auto insurance churn dataset comes from Kaggle, hence all of the details such as background of data, description of data, and problem to be solved can be found in that open-source platform.

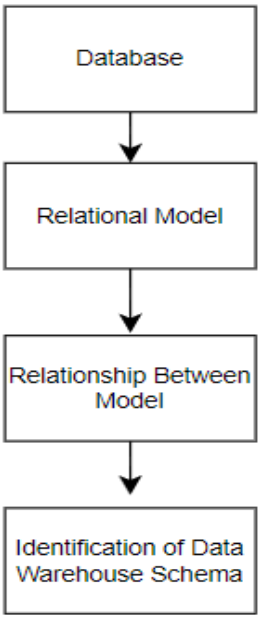


Figure 2.4 Flow of process of Database

Figure 2.4 represents flow of process of the database including relational model, relationship between model and identification of data warehouse schema. In our project, PowerBI is utilized to create relational models. Then, we undergo the process of ETL (Extract, Transform and Load) pipeline by using PostgreSQL and Jupyter Notebook. The raw data was extracted in PgAdmin and connected into the Jupyter Notebook for data cleaning. After cleaning the raw data, it is needed to load again that data into PgAdmin for further data analysis such as OLAP operations and data visualization.

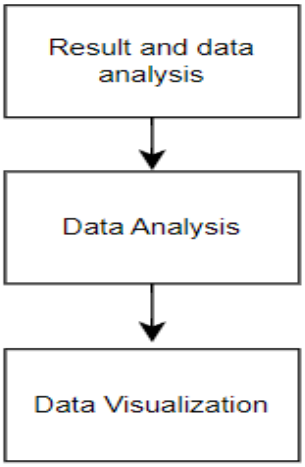


Figure 2.5 Flow of process of Results and Data Analysis

Figure 2.5 shows the process of results and data analysis. In our project, we used PostgreSQL to do OLAP operations such as grouping sets, group by, roll up, slicing, dicing for data analysis. After we finished the OLAP operations, we utilized PowerBI to do data visualization for the purpose to uncover hidden insights and relationships between data.

3.0 DATABASE

3.1 Relational Model

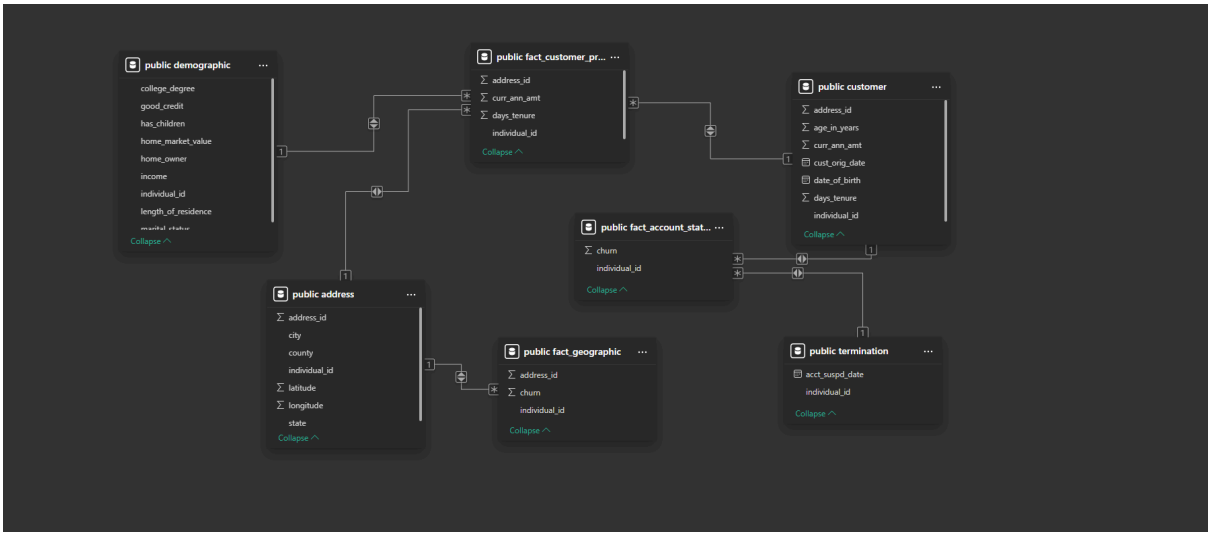


Figure 3.1 Relational Model using Power BI

Figure 3.1 is a relational model created by using Power BI software. The relational model can view the relationship between the data.

3.2 Relational between Data

Data	Relationship
fact_customer_profile → demographic	Many to one
fact_customer_profile → address	Many to one
fact_customer_profile → customer	Many to one
address → fact_geographic	One to many
customer → fact_account_status	One to many
fact_account_status → termination	Many to one

Table 3.1 : Relational between Data

3.3 Identification of Data Warehouse Schema

Based on Figure 3.1, the data warehouse schema represented by the fact_customer_profile dataset is **Galaxy Schema**. Through this dataset, it is performed by 3 fact tables which are **fact_customer_profile**, **fact_account_status**, and **fact_geographic**. They also consist of 4 dimensional tables which are **demographic**, **address**, **customer**, and **termination**.

4.0 ETL PIPELINE

4.1 ETL Pipeline

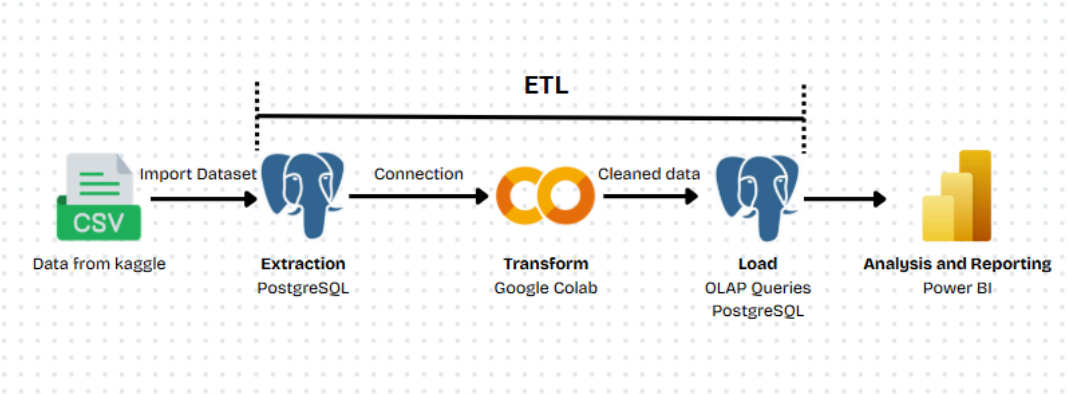


Figure 4.1 : ETL Pipeline

ETL pipeline has been shown in Figure 4.1. From Figure 4.1, we can know that the whole process of ETL is utilized by the application of PostgreSQL and also Google Collab. ETL is a process of extracting data from any open and trustworthy source, then occurring the process of data preparation such as data cleaning before we load data into PostgreSQL.In this project, we utilized PostgreSQL to extract our data by creating a database and tables. After that, we install some necessary packages before we do data cleaning by using Python in the Google Collab. After we clean all of the useless data, we load the cleaned data into PostgreSQL. OLAP operations such as slicing,dicing,roll up, grouping sets are completed in PostgreSQL to get results for analysis. Finally, we visualize data by using PowerBI to uncover hidden insights and trends represented by data.

4.2 ETL Process

4.2.1 Extract

When starting the ETL process, it is essential to create a new database and new tables which are going to be used to store raw data

Creating New Database

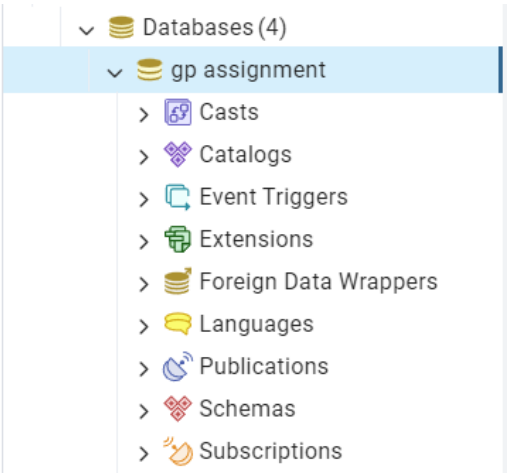


Figure 4.2.1 : New Database Created

Create New Tables

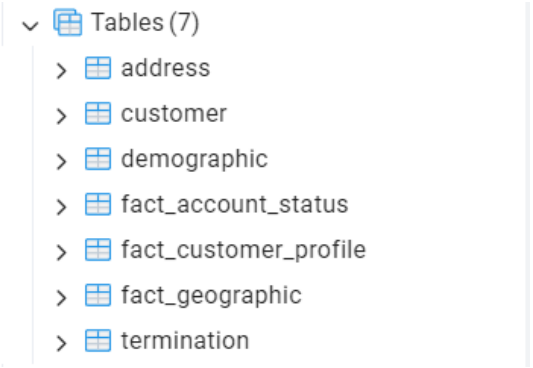


Figure 4.2.2 : 7 New Tables Created

Full coding to create the 7 tables

```
CREATE TABLE address (
        individual_id NUMERIC(20,0) PRIMARY KEY,
   address_id NUMERIC (20,0),
   latitude NUMERIC,
   longitude NUMERIC,
   city TEXT,
   state TEXT,
   county TEXT
);

CREATE TABLE customer (
   individual_id NUMERIC(20,0) PRIMARY KEY,
   address_id NUMERIC,
   curr_ann_amt NUMERIC,
   days_tenure NUMERIC,
   cust_orig_date TEXT,
   age_in_years NUMERIC,
   date_of_birth TEXT
);

CREATE TABLE demographic (
   individual_id NUMERIC(20,0) PRIMARY KEY,
   income NUMERIC,
   has_children NUMERIC,
   length_of_residence NUMERIC,
   marital_status TEXT,
   home_market_value TEXT,
   home_owner NUMERIC,
   college_degree NUMERIC,
   good_credit NUMERIC
);
```

```
CREATE TABLE termination (
   individual_id NUMERIC(20,0) PRIMARY KEY,
   acct_suspd_date DATE
);

CREATE TABLE fact_customer_profile (
   individual_id     NUMERIC(20,0) PRIMARY KEY,
   address_id        NUMERIC (20,0),
   curr_ann_amt      NUMERIC(12, 2),
   days_tenure       NUMERIC (20,0),

   CONSTRAINT fk_customer
      FOREIGN KEY (individual_id)
      REFERENCES customer(individual_id),

   CONSTRAINT fk_address
      FOREIGN KEY (individual_id)
      REFERENCES address(individual_id),

         CONSTRAINT fk_demographic
                FOREIGN KEY (individual_id)
                REFERENCES demographic(individual_id)
);

CREATE TABLE fact_account_status (
   individual_id     NUMERIC(20,0) PRIMARY KEY,
   churn             NUMERIC,

   CONSTRAINT fk_termination
      FOREIGN KEY (individual_id)
      REFERENCES termination(individual_id),

   CONSTRAINT fk_customer_status
      FOREIGN KEY (individual_id)
```

```
        REFERENCES customer(individual_id)
);

CREATE TABLE fact_geographic (
    individual_id     NUMERIC(20,0) PRIMARY KEY,
    address_id        NUMERIC(20,0),
    churn             NUMERIC,


    CONSTRAINT fk_address_geo
        FOREIGN KEY (individual_id)
        REFERENCES address(individual_id)
);
```

Overview of data in all 7 tables

QUERY	RESULT

SELECT * FROM "address"

	individual_id [PK] numeric (20)	address_id numeric (20)	latitude numeric	longitude numeric	city text	state text	county text
1	221300017872	521300005982	32.578829	-96.305006	Kaufman	TX	Kaufman
2	221300134410	521300051115	32.732209	-97.000893	Grand Prairie	TX	Dallas
3	221300673028	521300247929	32.819776999999995	-96.846938	Dallas	TX	Dallas
4	221301573419	521300570147	32.684065	-97.16218	Arlington	TX	Tarrant
5	221301622347	521300588399	32.751398	-97.376745	Fort Worth	TX	Tarrant
6	221302735288	521300990369	[null]	[null]	Dallas	TX	Dallas
7	221300697686	521300256124	33.006985	-96.893972	Carrollton	TX	Denton
8	221302209400	521300798596	33.085901	-96.648596	Allen	TX	Collin
9	221302854567	521301084127	32.828468	-97.173721	Bedford	TX	Tarrant
10	221301247282	521300454553	32.753978000000004	-96.785564	Dallas	TX	Dallas
11	221301301436	521300474503	32.645107	-97.157319	Arlington	TX	Tarrant
12	221302005705	521300726252	33.089878	-96.874424	The Colony	TX	Denton
13	221302781834	521301011394	33.168202	-96.640185	Mckinney	TX	Collin
14	221301392462	521300507544	32.841713	-96.999883	Irving	TX	Dallas
15	221303096842	521301326403	32.851905	-96.808335	Dallas	TX	Dallas
16	221301479205	521300537439	32.675671	-97.364207	Fort Worth	TX	Tarrant
17	221303215002	521301444564	32.639399	-97.353364	Fort Worth	TX	Tarrant
18	221301003945	521300368024	32.807283	-96.598049	Mesquite	TX	Dallas
19	221300771454	521300283230	32.869218	-97.17209	Hurst	TX	Tarrant
20	221302382580	521300862858	32.841944	-97.374225	Fort Worth	TX	Tarrant
21	221300001751	521300000712	[null]	[null]	Fort Worth	TX	Tarrant

SELECT * FROM "customer"

	individual_id [PK] numeric (20)	address_id numeric	curr_ann_amt numeric	days_tenure numeric	cust_orig_date text	age_in_years numeric	date_of_birth text
1	221300017872	521300005982.0	818.877997	1454.0	2018-12-09	44	1978-06-23
2	221300134410	521300051115.0	974.199182	1795.0	2018-01-02	72	1950-05-30
3	221300673028	521300247929.0	967.375112	4818.0	2009-09-23	55	1967-07-07
4	221301573419	521300570147.0	992.409561	130.0	2022-07-25	53	1969-05-25
5	221301622347	521300588399.0	784.633494	5896.0	2006-10-11	50	1972-09-25
6	221302735288	521300990369.0	909.916163	484.0	2021-08-05	33	1990-04-20
7	221300697686	521300256124.0	1084.048271	2896.0	2014-12-28	71	1952-02-29
8	221302209400	521300798596.0	1277.370767	4096.0	2011-09-15	46	1976-12-24
9	221302854567	521301084127.0	917.201036	2107.0	2017-02-24	56	1966-07-27
10	221301247282	521300454553.0	1095.390747	3387.0	2013-08-24	64	1958-08-28
11	221301301436	521300474503.0	905.038011	5336.0	2008-04-23	67	1956-04-29
12	221302005705	521300726252.0	462.911054	6020.0	2006-06-09	41	1981-06-22
13	221302781834	521301011394.0	722.089567	374.0	2021-11-23	69	1953-10-29
14	221301392462	521300507544.0	841.02187	150.0	2022-07-05	59	1963-11-28
15	221303096842	521301326403.0	741.671055	4711.0	2010-01-08	44	1979-03-24
16	221301479205	521300537439.0	1398.728178	5049.0	2009-02-04	41	1981-12-23
17	221303215002	521301444564.0	913.663034	6226.0	2005-11-15	60	1962-05-27
18	221301003945	521300368024.0	438.245021	1018.0	2020-02-18	49	1974-01-22
19	221300771454	521300283230.0	953.30552	5463.0	2007-12-18	68	1955-02-28
20	221302382580	521300862858.0	725.165234	3336.0	2013-10-14	38	1984-09-22
21	221300001751	521300000712.0	716.719703	5654.0	2007-06-10	49	1974-01-22

SELECT * FROM "demographic"

	individual_id [PK] numeric (20)	income numeric	has_children numeric	length_of_residence numeric	marital_status text	home_market_value text	home_owner numeric	college_degree numeric	good_credit numeric
1	221300017872	22500.0	1.0	15.0	Married	50000 - 74999	1.0	1.0	1.0
2	221300134410	27500.0	0.0	2.0	Single	50000 - 74999	1.0	0.0	0.0
3	221300673028	42500.0	0.0	10.0	Married	75000 - 99999	1.0	0.0	0.0
4	221301573419	125000.0	1.0	6.0	Married	175000 - 199999	1.0	0.0	1.0
5	221301622347	87500.0	1.0	4.0	Married	225000 - 249999	1.0	1.0	1.0
6	221302735288	52500.0	0.0	2.0	Married	125000 - 149999	1.0	0.0	1.0
7	221300697686	125000.0	1.0	8.0	Married	100000 - 124999	1.0	0.0	1.0
8	221302209400	47500.0	0.0	12.0	Married	125000 - 149999	1.0	0.0	1.0
9	221302854567	47500.0	0.0	6.0	Single	50000 - 74999	1.0	1.0	1.0
10	221301247282	32500.0	0.0	15.0	Married	25000 - 49999	1.0	0.0	0.0
11	221301301436	125000.0	0.0	15.0	Married	125000 - 149999	1.0	1.0	1.0
12	221302005705	37500.0	1.0	6.0	Married	75000 - 99999	1.0	1.0	1.0
13	221302781834	62500.0	1.0	5.0	Single	100000 - 124999	1.0	1.0	1.0
14	221301392462	57500.0	0.0	11.0	Single	150000 - 174999	0.0	0.0	1.0
15	221303096842	125000.0	1.0	10.0	Single	225000 - 249999	1.0	0.0	1.0
16	221301479205	125000.0	1.0	15.0	Married	75000 - 99999	1.0	1.0	1.0
17	221303215002	47500.0	0.0	12.0	Married	50000 - 74999	1.0	0.0	1.0
18	221301003945	70000.0	0.0	6.0	Married	150000 - 174999	0.0	0.0	1.0
19	221300771454	125000.0	0.0	9.0	Married	75000 - 99999	1.0	0.0	1.0
20	221302382580	70000.0	0.0	7.0	Married	75000 - 99999	1.0	0.0	1.0
21	221300001751	87500.0	1.0	0.0	Single	[null]	0.0	0.0	0.0

SELECT * FROM "termination"

	individual_id [PK] numeric (20)	acct_suspd_date date
1	221300017872	[null]
2	221300134410	[null]
3	221300673028	[null]
4	221301573419	2021-12-22
5	221301622347	[null]
6	221302735288	[null]
7	221300697686	[null]
8	221302209400	[null]
9	221302854567	2022-12-01
10	221301247282	[null]
11	221301301436	[null]
12	221302005705	[null]
13	221302781834	[null]
14	221301392462	[null]
15	221303096842	[null]
16	221301479205	[null]
17	221303215002	[null]
18	221301003945	[null]
19	221300771454	[null]
20	221302382580	[null]
21	221300001751	2021-12-20

SELECT * FROM "fact_customer_profile"

	individual_id [PK] numeric (20)	address_id numeric (20)	curr_ann_amt numeric (12,2)	days_tenure numeric (20)
1	221300017872	521300005982	818.88	1454
2	221300134410	521300051115	974.20	1795
3	221300673028	521300247929	967.38	4818
4	221301573419	521300570147	992.41	130
5	221301622347	521300588399	784.63	5896
6	221302735288	521300990369	909.92	484
7	221300697686	521300256124	1084.05	2896
8	221302209400	521300798596	1277.37	4096
9	221302854567	521301084127	917.20	2107
10	221301247282	521300454553	1095.39	3387
11	221301301436	521300474503	905.04	5336
12	221302005705	521300726252	462.91	6020
13	221302781834	521301011394	722.09	374
14	221301392462	521300507544	841.02	150
15	221303096842	521301326403	741.67	4711
16	221301479205	521300537439	1398.73	5049
17	221303215002	521301444564	913.66	6226
18	221301003945	521300368024	438.25	1018
19	221300771454	521300283230	953.31	5463
20	221302382580	521300862858	725.17	3336
21	221300001751	521300000712	716.72	5654

SELECT * FROM "fact_account_status"

	individual_id [PK] numeric (20)	churn numeric
1	221300017872	0
2	221300134410	0
3	221300673028	0
4	221301573419	1
5	221301622347	0
6	221302735288	0
7	221300697686	0
8	221302209400	0
9	221302854567	1
10	221301247282	0
11	221301301436	0
12	221302005705	0
13	221302781834	0
14	221301392462	0
15	221303096842	0
16	221301479205	0
17	221303215002	0
18	221301003945	0
19	221300771454	0
20	221302382580	0
21	221300001751	1

SELECT * FROM "fact_geographic"

	individual_id [PK] numeric (20)	address_id numeric (20)	churn numeric
1	221300017872	521300005982	0
2	221300134410	521300051115	0
3	221300673028	521300247929	0
4	221301573419	521300570147	1
5	221301622347	521300588399	0
6	221302735288	521300990369	0
7	221300697686	521300256124	0
8	221302209400	521300798596	0
9	221302854567	521301084127	1
10	221301247282	521300454553	0
11	221301301436	521300474503	0
12	221302005705	521300726252	0
13	221302781834	521301011394	0
14	221301392462	521300507544	0
15	221303096842	521301326403	0
16	221301479205	521300537439	0
17	221303215002	521301444564	0
18	221301003945	521300368024	0
19	221300771454	521300283230	0
20	221302382580	521300862858	0
21	221300001751	521300000712	1

After all the data has been extracted into PostgreSQL, a connection is needed to proceed with the transformation stage by using Jupyter Notebook. First, install necessary packages in Anaconda Prompt as shown below :

```
!pip install ipython-sql
!pip install sqlalchemy
!pip install psycopg2
!pip install python-sql
!pip install pandas-sql
!pip install sql-queries
!pip install missingno
```

Figure shows downloading of necessary libraries

The ipython-sql package enables users to execute SQL statements inside Jupyter Notebook platforms. This package provides robust features together with Pandas support which enables users to perform database operations and analyze data through notebook interactions. The sqlalchemy framework functions as a complete Python SQL toolkit which allows developers to access SQL databases through an object-oriented Python interface. The project employs psycopg2 for linking Python with PostgreSQL databases to enable table access and record updates and data refresh functions. Through the pandasql package, users can query Pandas

DataFrames because it detects them automatically and supports standard SQL queries. The dataset evaluation process uses missingno to detect and display patterns of missing data within the dataset.

```
%reload_ext sql
```

Figure shows loading of ipython_sql

```
from sqlalchemy import create_engine
```

Figure shows creating engine from sqlalchemy

```
import pandas as pd
import psycopg2 as ps
import pandas.io.sql as sqlio
import missingno as msno
```

Figure shows importing necessary libraries needed for ETL process

```
connectpg=ps.connect(dbname="gp assignment ",
                     user="postgres", password="1234", host="localhost", port="5432")
```

Figure shows connection of PgAdmin with Jupyter Notebook

```
# Connect to your PostgreSQL database
engine = create_engine('postgresql://postgres:1234@localhost:5432/gp assignment ')

# Step 1: Get all tables from the public schema
tables = pd.read_sql("""
    SELECT tablename
    FROM pg_catalog.pg_tables
    WHERE schemaname = 'public'
""", engine)
```

Figure shows extraction of all the tables in PgAdmin database

4.2.2 Transform

Once the raw data is connected to the Jupyter Notebook, data cleaning becomes a crucial step in preparing the data for further analysis. Initially, we installed the necessary connectors to ensure seamless data transfer from PostgreSQL to Jupyter Notebook. During the data cleaning process, we can store the data in a DataFrame using the pandas library.

After we connect and extract data from PgAdmin into Jupyter Notebook, we perform the process of data cleaning to identify any null values and dirty data from each table and remove them to make raw data become clean data.

```
:
import pandas as pd
from sqlalchemy import create_engine

# Connect to your PostgreSQL database
engine = create_engine('postgresql://postgres:1234@localhost:5432/gp assignment ')

# Step 1: Get all tables from the public schema
tables = pd.read_sql("""
    SELECT tablename
    FROM pg_catalog.pg_tables
    WHERE schemaname = 'public'
""", engine)

# Step 2: Loop through tables and check for nulls
for table in tables['tablename']:
    print(f"\n🔍 Missing Values in Table: {table}")
    df = pd.read_sql(f'SELECT * FROM "{table}"', engine)  # Use quotes in case of lowercase/mixed-case names
    print(df.isnull().sum())
```

🔍 Missing Values in Table: fact_customer_profile
```
individual_id    0
address_id       0
curr_ann_amt     0
days_tenure      0
dtype: int64
```

🔍 Missing Values in Table: customer
```
individual_id     0
address_id        0
curr_ann_amt      0
days_tenure       0
cust_orig_date    0
age_in_years      0
date_of_birth     0
dtype: int64
```

🔍 Missing Values in Table: termination
```
individual_id           0
acct_suspd_date    1487453
dtype: int64
```

🔍 Missing Values in Table: fact_geographic
```
individual_id    0
address_id       0
churn            0
dtype: int64
```

🔍 Missing Values in Table: demograph
```
individual_id              0
income                     0
has_children               0
length_of_residence        0
marital_status             0
home_market_value      92286
home_owner                 0
college_degree             0
good_credit                0
dtype: int64
```

🔍 Missing Values in Table: address
```
individual_id         0
address_id            0
latitude         253719
longitude        253719
city              12067
state                 0
county            12067
dtype: int64
```

We began the data cleaning process by addressing the home_market_value column in the **demographic** table. This column originally contained data in a string format representing a range (e.g., "300000 - 500000"). To standardize and simplify the data for analysis, we extracted the minimum value from each range using regular expressions. The extracted values were then converted to integer data type. For missing values, we imputed them with 0, assuming the homeowner may not own a property or the value is otherwise unavailable.

```
# Step 1: Fill missing values with '0'
demographic_df['home_market_value'] = demographic_df['home_market_value'].fillna('0')

# Step 2: Extract the first number using regex
demographic_df['home_market_value'] = demographic_df['home_market_value'].str.extract(r'(\d+)')

# Step 3: Convert to integer
demographic_df['home_market_value'] = demographic_df['home_market_value'].astype(int)
```

Output :

	individual_id	income	has_children	length_of_residence	marital_status	home_market_value	home_owner	college_degree	good_credit
0	2.213000e+11	22500.0	1.0	15.0	Married	50000	1.0	1.0	1.0
1	2.213001e+11	27500.0	0.0	2.0	Single	50000	1.0	0.0	0.0
2	2.213007e+11	42500.0	0.0	10.0	Married	75000	1.0	0.0	0.0
3	2.213016e+11	125000.0	1.0	6.0	Married	175000	1.0	0.0	1.0
4	2.213016e+11	87500.0	1.0	4.0	Married	225000	1.0	1.0	1.0

In the **address** table, we identified null values in the city,county, longitude and latitude columns. Since these fields contain categorical and geographic information that cannot be reliably imputed (e.g., they are not numeric or estimable), we decided to remove the corresponding rows entirely. This ensures the dataset maintains integrity and avoids introducing inaccurate or misleading data.

```
# Load address table from SQL
address_df = pd.read_sql('SELECT * FROM address', engine)

# Drop rows where 'city' or 'county' is missing
address_cleaned = address_df.dropna(subset=['city', 'county'])

# Drop the entire columns 'latitude' and 'longitude' (since they contain many missing values)
address_cleaned = address_cleaned.dropna(subset=['latitude', 'longitude'])

# Check missing values again
address_cleaned.isnull().sum()
```

```
individual_id    0
address_id       0
city             0
state            0
county           0
dtype: int64
```

Next, in the **termination** table, we encountered null values in the acct_suspd_date column. These missing values indicate that the accounts have not been suspended yet. To handle this, we replaced the nulls with a placeholder date of **2099-12-31**, which clearly signifies accounts that are still active. This approach allows for consistent data formatting and easier filtering during analysis. We also documented this assumption to ensure clarity in interpretation.

```
# Load termination table
termination_df = pd.read_sql('SELECT * FROM termination', engine)

# Fill missing acct_suspd_date with a placeholder date
termination_df['acct_suspd_date'] = termination_df['acct_suspd_date'].fillna(pd.to_datetime('2099-12-31'))
```

After handling all missing values either by removal or imputation we proceeded to balance the number of rows across all tables. Since the data cleaning was performed on a table-by-table basis, discrepancies in row counts emerged due to varying amounts of missing or invalid data in each table.

To ensure consistency and avoid potential errors during table joins or when loading the cleaned data back into **pgAdmin (PostgreSQL)** for future analysis, we standardized the number of rows across tables. Maintaining the same number of rows is important, especially when the tables are related by primary or foreign keys, as mismatched row counts can lead to issues during table joins or data integration.

We decided to use the **address** table as the reference (or **primary anchor**) for aligning and filtering the other tables. This decision was based on the fact that the **address** table had the highest number of rows removed during the data cleaning process due to missing values in critical fields like city, county, longitude and latitude.

By using the address table as the reference, we ensured that all other tables were filtered to match its cleaned row count, maintaining consistency across the dataset. This approach helps preserve referential integrity and prevents issues when performing joins or further analysis.

```python
# Step 1: Get valid individual_ids from the already cleaned address
valid_ids = address_cleaned['individual_id'].unique()

# Step 2: Load other tables
customer_df = pd.read_sql('SELECT * FROM customer', engine)
fact_df = pd.read_sql('SELECT * FROM fact_customer_profile', engine)
demo_df = pd.read_sql('SELECT * FROM demographic', engine)
term_df = pd.read_sql('SELECT * FROM termination', engine)
acc_df = pd.read_sql('SELECT * FROM fact_account_status', engine)
geo_df = pd.read_sql('SELECT * FROM fact_geographic', engine)

# Step 3: Filter all other tables using valid individual_id from address_cleaned
customer_filtered = customer_df[customer_df['individual_id'].isin(valid_ids)]
fact_filtered = fact_df[fact_df['individual_id'].isin(valid_ids)]
demo_filtered = demo_df[demo_df['individual_id'].isin(valid_ids)]
term_filtered = term_df[term_df['individual_id'].isin(valid_ids)]
acc_filtered = acc_df[acc_df['individual_id'].isin(valid_ids)]
geo_filtered = geo_df[geo_df['individual_id'].isin(valid_ids)]

# Step 4: Check row counts
print("✅ Filtered row counts based on cleaned address:")
print("address_cleaned:", len(address_cleaned))
print("customer_filtered:", len(customer_filtered))
print("fact_filtered:", len(fact_filtered))
print("demo_filtered:", len(demo_filtered))
print("term_filtered:", len(term_filtered))
print("acc_filtered:", len(acc_filtered))
print("geo_filtered:", len(geo_filtered))
```

Output

```
✅ Filtered row counts based on cleaned address:
address_cleaned: 1416734
customer_filtered: 1416734
fact_filtered: 1416734
demo_filtered: 1416734
term_filtered: 1416734
acc_filtered: 1416734
geo_filtered: 1416734
```

After aligning all tables based on the **address** table as the reference, we conducted a final check for missing values in each table. This step was crucial to ensure that no overlooked null values remained after the initial cleaning and row alignment process. By verifying the completeness of each dataset, we aimed to maintain data quality and integrity, minimizing the risk of errors during future analysis or when re-importing the data into **pgAdmin**.

```
# Dictionary of all _filtered tables
filtered_tables = {
    'customer_filtered': customer_filtered,
    'fact_filtered': fact_filtered,
    'demo_filtered': demo_filtered,
    'term_filtered': term_filtered,
    'acc_filtered': acc_filtered,
    'geo_filtered': geo_filtered
}

# Loop through and display missing values
for name, df in filtered_tables.items():
    print(f"\n🔍 Missing Values in Table: {name}")
    print(df.isnull().sum())
```

🔍 Missing Values in Table: customer_filtered

```
individual_id     0
address_id        0
curr_ann_amt      0
days_tenure       0
cust_orig_date    0
age_in_years      0
date_of_birth     0
dtype: int64
```

🔍 Missing Values in Table: fact_filtered

```
individual_id    0
address_id       0
curr_ann_amt     0
days_tenure      0
dtype: int64
```

🔍 Missing Values in Table: demo_filtered

```
individual_id          0
income                 0
has_children           0
length_of_residence    0
marital_status         0
home_market_value      0
home_owner             0
college_degree         0
good_credit            0
dtype: int64
```

🔍 Missing Values in Table: term_filtered

```
individual_id      0
acct_suspd_date    0
dtype: int64
```

🔍 Missing Values in Table: acc_filtered

```
individual_id    0
churn            0
dtype: int64
```

🔍 Missing Values in Table: geo_filtered

```
individual_id    0
address_id       0
churn            0
dtype: int64
```

After completing the data cleaning and verification process, we exported all the cleaned tables as CSV files. This format was chosen for its compatibility with various data analysis tools and ease of re-importing into pgAdmin or other database systems for future analysis. Saving the files in CSV format also ensures the cleaned data is well-documented and readily accessible for subsequent processing or sharing.

```
# Dictionary of filtered DataFrames (excluding address)
filtered_tables = {
    'customer_filtered': customer_filtered,
    'fact_filtered': fact_filtered,
    'demo_filtered': demo_filtered,
    'term_filtered': term_filtered,
    'acc_filtered': acc_filtered,
    'geo_filtered': geo_filtered
}

# Export each DataFrame to a CSV file
for name, df in filtered_tables.items():
    filename = f"{name}.csv"
    df.to_csv(filename, index=False)
    print(f"✅ Saved: {filename}")
```

```
✅ Saved: customer_filtered.csv
✅ Saved: fact_filtered.csv
✅ Saved: demo_filtered.csv
✅ Saved: term_filtered.csv
✅ Saved: acc_filtered.csv
✅ Saved: geo_filtered.csv
```

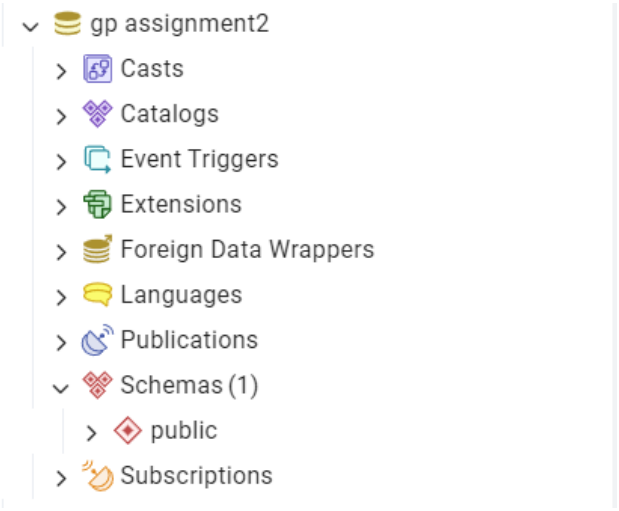
```
address_cleaned.to_csv("address_cleaned.csv", index=False)
```

4.2.3 Load

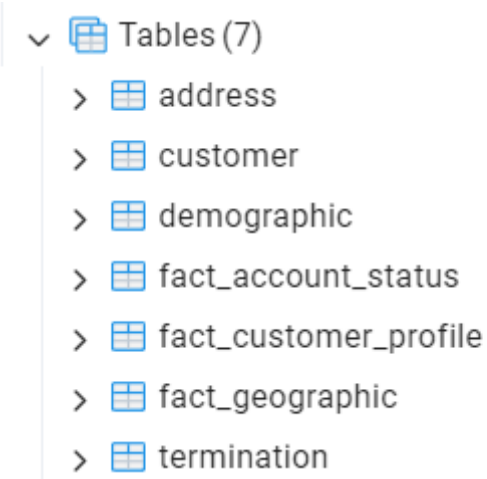
After the process of data cleaning is completed using Jupyter Notebook, 7 new CSV files will be created which will be needed to load into PostgreSQL for further progress.

In the loading process, we will need to create a new database which is named by "gp assignment2" and also 7 new tables for our cleaned data. After that, we import again the cleaned csv files into PostgreSQL.

Create a new database "gp assignment2" to load the data after cleaning the raw data using Jupyter Notebook



Create 7 new tables to load the cleaned data into PostgreSQL



Full coding to create the tables shown above

```
CREATE TABLE address (
        individual_id NUMERIC(20,0) PRIMARY KEY,
    address_id NUMERIC (20,0),
    latitude NUMERIC,
    longitude NUMERIC,
    city TEXT,
    state TEXT,
    county TEXT
);

CREATE TABLE customer (
    individual_id NUMERIC(20,0) PRIMARY KEY,
    address_id NUMERIC,
    curr_ann_amt NUMERIC,
    days_tenure NUMERIC,
    cust_orig_date TEXT,
    age_in_years NUMERIC,
    date_of_birth TEXT
);

CREATE TABLE demographic (
    individual_id NUMERIC(20,0) PRIMARY KEY,
    income NUMERIC,
    has_children NUMERIC,
    length_of_residence NUMERIC,
    marital_status TEXT,
    home_market_value TEXT,
    home_owner NUMERIC,
    college_degree NUMERIC,
    good_credit NUMERIC
);
```

```
CREATE TABLE termination (
   individual_id NUMERIC(20,0) PRIMARY KEY,
   acct_suspd_date DATE
);

CREATE TABLE fact_customer_profile (
   individual_id     NUMERIC(20,0) PRIMARY KEY,
   address_id        NUMERIC (20,0),
   curr_ann_amt      NUMERIC(12, 2),
   days_tenure       NUMERIC (20,0),

   CONSTRAINT fk_customer
      FOREIGN KEY (individual_id)
      REFERENCES customer(individual_id),

   CONSTRAINT fk_address
      FOREIGN KEY (individual_id)
      REFERENCES address(individual_id),

         CONSTRAINT fk_demographic
                FOREIGN KEY (individual_id)
                REFERENCES demographic(individual_id)
);

CREATE TABLE fact_account_status (
   individual_id     NUMERIC(20,0) PRIMARY KEY,
   churn             NUMERIC,

   CONSTRAINT fk_termination
      FOREIGN KEY (individual_id)
      REFERENCES termination(individual_id),

   CONSTRAINT fk_customer_status
      FOREIGN KEY (individual_id)
```

```
    REFERENCES customer(individual_id)
);

CREATE TABLE fact_geographic (
   individual_id     NUMERIC(20,0) PRIMARY KEY,
   address_id        NUMERIC(20,0),
   churn             NUMERIC,


   CONSTRAINT fk_address_geo
      FOREIGN KEY (individual_id)
      REFERENCES address(individual_id)
);
```

Overview of cleaned data

QUERY	RESULT
SELECT * FROM "address"	

	individual_id [PK] numeric (20)	address_id numeric (20)	latitude numeric	longitude numeric	city text	state text	county text
1	221300017872	521300005982	32.578829	-96.305006	Kaufman	TX	Kaufman
2	221300134410	521300051115	32.732209	-97.000893	Grand Prairie	TX	Dallas
3	221300673028	521300247929	32.819776999999995	-96.846938	Dallas	TX	Dallas
4	221301573419	521300570147	32.684065	-97.16218	Arlington	TX	Tarrant
5	221301622347	521300588399	32.751398	-97.376745	Fort Worth	TX	Tarrant
6	221300697686	521300256124	33.006985	-96.893972	Carrollton	TX	Denton
7	221302209400	521300798596	33.085901	-96.648596	Allen	TX	Collin
8	221302854567	521301084127	32.828468	-97.173721	Bedford	TX	Tarrant
9	221301247282	521300454553	32.753978000000004	-96.785564	Dallas	TX	Dallas
10	221301301436	521300474503	32.645107	-97.157319	Arlington	TX	Tarrant
11	221302005705	521300726252	33.089878	-96.874424	The Colony	TX	Denton
12	221302781834	521301011394	33.168202	-96.640185	Mckinney	TX	Collin
13	221301392462	521300507544	32.841713	-96.999883	Irving	TX	Dallas
14	221303096842	521301326403	32.851905	-96.808335	Dallas	TX	Dallas
15	221301479205	521300537439	32.675671	-97.364207	Fort Worth	TX	Tarrant
16	221303215002	521301444564	32.639399	-97.353364	Fort Worth	TX	Tarrant
17	221301003945	521300368024	32.807283	-96.598049	Mesquite	TX	Dallas
18	221300771454	521300283230	32.869218	-97.17209	Hurst	TX	Tarrant
19	221302382580	521300862858	32.841944	-97.374225	Fort Worth	TX	Tarrant
20	221302855707	521301085267	32.924986999999994	-96.769758	Dallas	TX	Dallas
21	221300623694	521300230809	32.72473	-96.97322	Grand Prairie	TX	Dallas

SELECT * FROM "customer"

	individual_id [PK] numeric (20)	address_id numeric	curr_ann_amt numeric	days_tenure numeric	cust_orig_date text	age_in_years numeric	date_of_birth text
1	221300017872	521300005982.0	818.877997	1454.0	2018-12-09	44.0	1978-06-23
2	221300134410	521300051115.0	974.199182	1795.0	2018-01-02	72.0	1950-05-30
3	221300673028	521300247929.0	967.375112	4818.0	2009-09-23	55.0	1967-07-07
4	221301573419	521300570147.0	992.409561	130.0	2022-07-25	53.0	1969-05-25
5	221301622347	521300588399.0	784.633494	5896.0	2006-10-11	50.0	1972-09-25
6	221300697686	521300256124.0	1084.048271	2896.0	2014-12-28	71.0	1952-02-29
7	221302209400	521300798596.0	1277.370767	4096.0	2011-09-15	46.0	1976-12-24
8	221302854567	521301084127.0	917.201036	2107.0	2017-02-24	56.0	1966-07-27
9	221301247282	521300454553.0	1095.390747	3387.0	2013-08-24	64.0	1958-08-28
10	221301301436	521300474503.0	905.038011	5336.0	2008-04-23	67.0	1956-04-29
11	221302005705	521300726252.0	462.911054	6020.0	2006-06-09	41.0	1981-06-22
12	221302781834	521301011394.0	722.089567	374.0	2021-11-23	69.0	1953-10-29
13	221301392462	521300507544.0	841.02187	150.0	2022-07-05	59.0	1963-11-28
14	221303096842	521301326403.0	741.671055	4711.0	2010-01-08	44.0	1979-03-24
15	221301479205	521300537439.0	1398.728178	5049.0	2009-02-04	41.0	1981-12-23
16	221303215002	521301444564.0	913.663034	6226.0	2005-11-15	60.0	1962-05-27
17	221301003945	521300368024.0	438.245021	1018.0	2020-02-18	49.0	1974-01-22
18	221300771454	521300283230.0	953.30552	5463.0	2007-12-18	68.0	1955-02-28
19	221302382580	521300862858.0	725.165234	3336.0	2013-10-14	38.0	1984-09-22
20	221302855707	521301085267.0	908.895897	6291.0	2005-09-11	54.0	1969-03-25
21	221300623694	521300230809.0	885.991042	662.0	2021-02-08	108.0	1914-05-09

SELECT * FROM "demographic"

	individual_id [PK] numeric (20)	income numeric	has_children numeric	length_of_residence numeric	marital_status text	home_market_value text	home_owner numeric	college_degree numeric	good_credit numeric
1	221300017872	22500.0	1.0	15.0	Married	50000 - 74999	1.0	1.0	1.0
2	221300134410	27500.0	0.0	2.0	Single	50000 - 74999	1.0	0.0	0.0
3	221300673028	42500.0	0.0	10.0	Married	75000 - 99999	1.0	0.0	0.0
4	221301573419	125000.0	1.0	6.0	Married	175000 - 199999	1.0	0.0	1.0
5	221301622347	87500.0	1.0	4.0	Married	225000 - 249999	1.0	1.0	1.0
6	221300697686	125000.0	1.0	8.0	Married	100000 - 124999	1.0	0.0	1.0
7	221302209400	47500.0	0.0	12.0	Married	125000 - 149999	1.0	0.0	1.0
8	221302854567	47500.0	0.0	6.0	Single	50000 - 74999	1.0	1.0	1.0
9	221301247282	32500.0	0.0	15.0	Married	25000 - 49999	1.0	0.0	0.0
10	221301301436	125000.0	0.0	15.0	Married	125000 - 149999	1.0	1.0	1.0
11	221302005705	37500.0	1.0	6.0	Married	75000 - 99999	1.0	1.0	1.0
12	221302781834	62500.0	1.0	5.0	Single	100000 - 124999	1.0	1.0	1.0
13	221301392462	57500.0	0.0	11.0	Single	150000 - 174999	0.0	0.0	1.0
14	221303096842	125000.0	1.0	10.0	Single	225000 - 249999	1.0	0.0	1.0
15	221301479205	125000.0	1.0	15.0	Married	75000 - 99999	1.0	1.0	1.0
16	221303215002	47500.0	0.0	12.0	Married	50000 - 74999	1.0	0.0	1.0
17	221301003945	70000.0	0.0	6.0	Married	150000 - 174999	0.0	0.0	1.0
18	221300771454	125000.0	0.0	9.0	Married	75000 - 99999	1.0	0.0	1.0
19	221302382580	70000.0	0.0	7.0	Married	75000 - 99999	1.0	0.0	1.0
20	221302855707	27500.0	1.0	15.0	Single	75000 - 99999	1.0	0.0	1.0
21	221300623694	125000.0	1.0	4.0	Married	75000 - 99999	1.0	0.0	1.0

SELECT * FROM "termination"

	individual_id [PK] numeric (20)	acct_suspd_date date
1	221300017872	2099-12-31
2	221300134410	2099-12-31
3	221300673028	2099-12-31
4	221301573419	2021-12-22
5	221301622347	2099-12-31
6	221300697686	2099-12-31
7	221302209400	2099-12-31
8	221302854567	2022-12-01
9	221301247282	2099-12-31
10	221301301436	2099-12-31
11	221302005705	2099-12-31
12	221302781834	2099-12-31
13	221301392462	2099-12-31
14	221303096842	2099-12-31
15	221301479205	2099-12-31
16	221303215002	2099-12-31
17	221301003945	2099-12-31
18	221300771454	2099-12-31
19	221302382580	2099-12-31
20	221302855707	2099-12-31
21	221300623694	2099-12-31

SELECT * FROM "fact_customer_profile"

	individual_id [PK] numeric (20)	address_id numeric (20)	curr_ann_amt numeric (12,2)	days_tenure numeric (20)
1	221300017872	521300005982	818.88	1454
2	221300134410	521300051115	974.20	1795
3	221300673028	521300247929	967.38	4818
4	221301573419	521300570147	992.41	130
5	221301622347	521300588399	784.63	5896
6	221300697686	521300256124	1084.05	2896
7	221302209400	521300798596	1277.37	4096
8	221302854567	521301084127	917.20	2107
9	221301247282	521300454553	1095.39	3387
10	221301301436	521300474503	905.04	5336
11	221302005705	521300726252	462.91	6020
12	221302781834	521301011394	722.09	374
13	221301392462	521300507544	841.02	150
14	221303096842	521301326403	741.67	4711
15	221301479205	521300537439	1398.73	5049
16	221303215002	521301444564	913.66	6226
17	221301003945	521300368024	438.25	1018
18	221300771454	521300283230	953.31	5463
19	221302382580	521300862858	725.17	3336
20	221302855707	521301085267	908.90	6291
21	221300623694	521300230809	885.99	662

SELECT * FROM "fact_account_status"

	individual_id [PK] numeric (20)	churn numeric
1	221300017872	0.0
2	221300134410	0.0
3	221300673028	0.0
4	221301573419	1.0
5	221301622347	0.0
6	221300697686	0.0
7	221302209400	0.0
8	221302854567	1.0
9	221301247282	0.0
10	221301301436	0.0
11	221302005705	0.0
12	221302781834	0.0
13	221301392462	0.0
14	221303096842	0.0
15	221301479205	0.0
16	221303215002	0.0
17	221301003945	0.0
18	221300771454	0.0
19	221302382580	0.0
20	221302855707	0.0
21	221300623694	0.0

SELECT * FROM "fact_geographic"

	individual_id [PK] numeric (20)	address_id numeric (20)	churn numeric
1	221300017872	521300005982	0.0
2	221300134410	521300051115	0.0
3	221300673028	521300247929	0.0
4	221301573419	521300570147	1.0
5	221301622347	521300588399	0.0
6	221300697686	521300256124	0.0
7	221302209400	521300798596	0.0
8	221302854567	521301084127	1.0
9	221301247282	521300454553	0.0
10	221301301436	521300474503	0.0
11	221302005705	521300726252	0.0
12	221302781834	521301011394	0.0
13	221301392462	521300507544	0.0
14	221303096842	521301326403	0.0
15	221301479205	521300537439	0.0
16	221303215002	521301444564	0.0
17	221301003945	521300368024	0.0
18	221300771454	521300283230	0.0
19	221302382580	521300862858	0.0
20	221302855707	521301085267	0.0
21	221300623694	521300230809	0.0

5.0 DATA ANALYSIS AND VISUALISATION

OLAP OPERATIONS

1. ROLL-UP

CODING :

```
SELECT
    a.county,
    a.city,
    COUNT(f.churn) AS churn_count
FROM fact_geographic f
JOIN address a ON f.address_id = a.address_id
WHERE f.churn = 1
GROUP BY ROLLUP(a.city, a.county)
HAVING NOT (a.city IS NULL OR a.county IS NULL)
ORDER BY churn_count DESC;
```

OUTPUT :

	county text	city text	churn_count bigint
1	Dallas	Dallas	38675
2	Tarrant	Fort Worth	30669
3	Tarrant	Arlington	14149
4	Collin	Plano	10325
5	Dallas	Garland	8972
6	Dallas	Irving	7114
7	Dallas	Grand Prairie	5933
8	Dallas	Mesquite	5732
9	Collin	Mckinney	5170
10	Dallas	Richardson	4404
11	Denton	Denton	4259
12	Collin	Allen	3619
13	Tarrant	Keller	3411
14	Denton	Lewisville	3348
15	Denton	Flower Mound	2883
16	Denton	Carrollton	2863
17	Denton	Frisco	2612
18	Dallas	Rowlett	2537
19	Johnson	Burleson	2475
20	Tarrant	North Richland Hills	2326
21	Rockwall	Rockwall	2237

Interpretation:

The first OLAP operation is the roll-up operation, which allows us to see the number of customers churn by city and zooming out to county. The table lists every city with its respective counties beside it. It is arranged in ascending order of the churn count. Here, we can see that Dallas city has the highest number of churned customers (38675) among other counties. It is also observed that there are two cities from Tarrant county that are in the top 3 highest churn. Therefore, these locations need to be focused on by the company to see why customers prefer to change their insurance agency.

2. DRILL-DOWN

CODING :

```
SELECT
    TO_CHAR(c.cust_orig_date::DATE, 'YYYY-MM') AS year_month_joined,
    COUNT(*) AS churned_customers
FROM customer c
JOIN fact_account_status f ON c.individual_id = f.individual_id
WHERE f.churn = 1
  AND c.cust_orig_date IS NOT NULL
  AND EXTRACT(YEAR FROM c.cust_orig_date::DATE) IN (2021, 2022)
GROUP BY TO_CHAR(c.cust_orig_date::DATE, 'YYYY-MM')
ORDER BY year_month_joined;
```

OUTPUT :

	year_month_joined text	churned_customers bigint
1	2021-01	165
2	2021-02	176
3	2021-03	163
4	2021-04	179
5	2021-05	168
6	2021-06	188
7	2021-07	177
8	2021-08	291
9	2021-09	463
10	2021-10	597
11	2021-11	674
12	2021-12	775
13	2022-01	933
14	2022-02	1117
15	2022-03	1857
16	2022-04	2218
17	2022-05	3280
18	2022-06	13726
19	2022-07	16179
20	2022-08	15003
21	2022-09	15672

Interpretation:

Next, the drill–down operation where we can see the number of churned customers every year, and zooming in by months. Tables show the churned customers in the last 2 years which are 2021 and 2022. Generally, it can be observed that there is a significant increase in the number of churns from 2021 to 2022. Starting from June 2022, the numbers rise up to 13726, way more than the month before. Therefore, insurance companies need to investigate why the numbers dramatically increase. It might be due to some controversies about the company, or there is lack of quality in customer service over the years.

3. SLICE

CODING :

```
SELECT d.good_credit, COUNT(f.churn)
FROM demographic d
JOIN fact_account_status f ON d.individual_id = f.individual_id
WHERE f.churn = 1
GROUP BY d.good_credit;
```

OUTPUT :

	good_credit numeric	count bigint
1	0.0	24109
2	1.0	137019

Interpretation:

From the slice operation, we can see that among the customers that have churned, there are more customers with higher credit scores rather than lower ones. Higher credit scores means that the customer has lower risks of accident. High credit customers can receive lower insurance premiums, paying less than the coverage.

Some logical justification for the churn numbers is, there might be competitors that offer more handsome rewards to customers with high credit scores. So, an analysis on potential competitors needs to be done in order to offer better premiums to customers with good credits, and reduce the churn numbers.

4. DICE

CODING :

```
SELECT
    c.individual_id,
    c.age_in_years,
    d.income,
    d.home_owner,
    a.city,
    f.churn
FROM customer c
JOIN demographic d ON c.individual_id = d.individual_id
JOIN address a ON c.address_id = a.address_id
JOIN fact_account_status f ON c.individual_id = f.individual_id
WHERE
    c.age_in_years < 30
    AND d.income < 35000
    AND d.home_owner = 0
    AND a.state = 'TX'
    AND f.churn = 1
ORDER BY income ASC;
```

OUTPUT :

	individual_id numeric (20)	age_in_years numeric	income numeric	home_owner numeric	city text	churn numeric
1	221302522878	28.0	5000.0	0.0	Denton	1.0
2	221300813747	26.0	5000.0	0.0	Denton	1.0
3	221302824806	29.0	5000.0	0.0	Fort Worth	1.0
4	221301452282	25.0	5000.0	0.0	Dallas	1.0
5	221302598925	27.0	5000.0	0.0	Fort Worth	1.0
6	221301860442	29.0	5000.0	0.0	Grand Prairie	1.0
7	221300842319	28.0	5000.0	0.0	Dallas	1.0
8	221301452748	27.0	5000.0	0.0	Fort Worth	1.0
9	221302046014	29.0	5000.0	0.0	Dallas	1.0
10	221300050814	26.0	5000.0	0.0	Dallas	1.0
11	221302595655	27.0	5000.0	0.0	Dallas	1.0
12	221301327491	28.0	5000.0	0.0	Dallas	1.0
13	221301796751	28.0	5000.0	0.0	Dallas	1.0
14	221301019132	29.0	5000.0	0.0	Dallas	1.0
15	221302598925	27.0	5000.0	0.0	Fort Worth	1.0
16	221302733860	29.0	5000.0	0.0	Dallas	1.0
17	221303131952	28.0	5000.0	0.0	Fort Worth	1.0
18	221302041132	27.0	5000.0	0.0	Fort Worth	1.0
19	221302001935	27.0	5000.0	0.0	Fort Worth	1.0
20	221302522878	28.0	5000.0	0.0	Denton	1.0
21	221300442551	28.0	5000.0	0.0	Dallas	1.0

Interpretation:

The dicing operation combines columns from multiple tables, where in this case, individual_id, age, income, home_owner (whether individual owns a home), city, and the data that shows if that individual churned from the company. The table shows all churned customers age 30 and below, income less than $35000 and does not own a home.

From the observations, we can see that there are many customers in these demographics who decided to change to another agency, with a majority of them coming from Dallas and Fort Worth. This could be a good indicator that customers below 30 years old, with relatively low income and renters, may have better options for their insurance package. Therefore, the company needs to offer more attractive packages and promotions to these groups, ensuring long-term customers in future.

5. PIVOT (BEFORE)

CODING :

```
SELECT
    d.marital_status,
    d.college_degree,
    COUNT(f.churn) AS churn_count
FROM demographic d
JOIN fact_account_status f ON d.individual_id = f.individual_id
WHERE f.churn = 1
GROUP BY d.marital_status, d.college_degree;
```

OUTPUT :

	marital_status text	college_degree numeric	churn_count bigint
1	Married	0.0	67033
2	Married	1.0	38105
3	Single	0.0	39580
4	Single	1.0	16410

PIVOT(AFTER)

CODING :

```
SELECT
    d.college_degree,
    SUM(f.churn) FILTER (WHERE marital_status = 'Married') AS Married,
    SUM(f.churn) FILTER (WHERE marital_status = 'Single') AS Single
FROM demographic d
JOIN fact_account_status f ON d.individual_id = f.individual_id
WHERE f.churn = 1
GROUP BY college_degree
ORDER BY college_degree;
```

OUTPUT :

	college_degree numeric	married numeric	single numeric
1	0.0	67033.0	39580.0
2	1.0	38105.0	16410.0

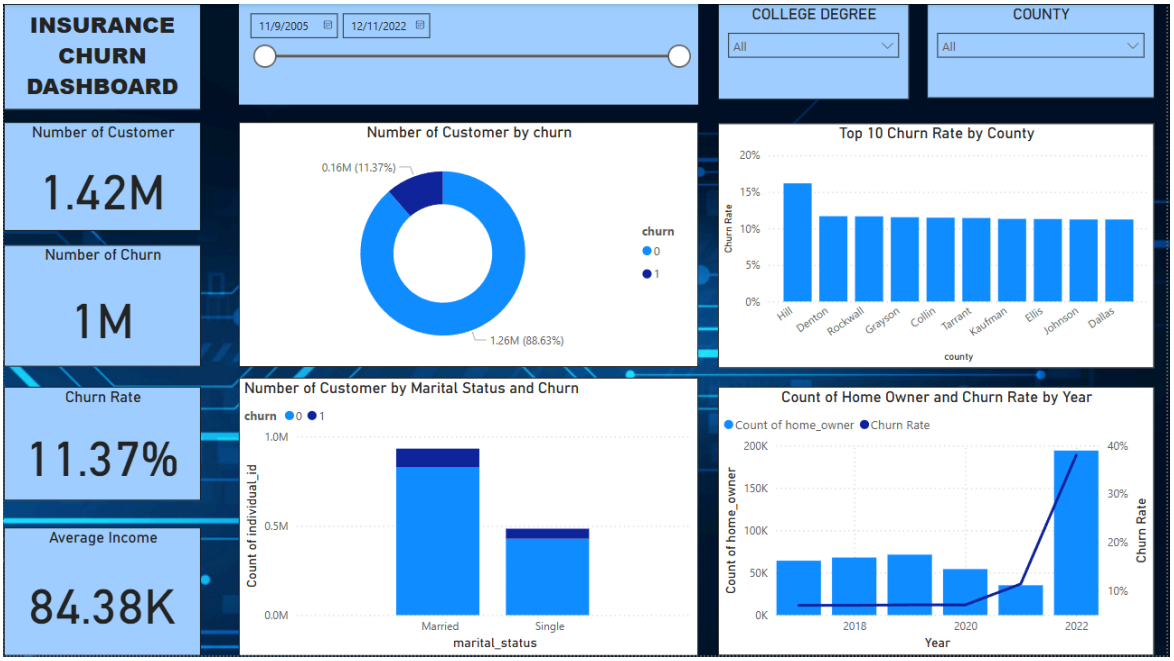
Interpretation:

The last operation is pivot, this operation allows us to change the view of the output shown. In this case, the row and column are reversed. The values show the number of churned customers from respective groups. Marital status, consisting of married or single, are changed from rows into separate columns. This allows a direct side-by-side comparison of churn counts for Married vs. Single customers, broken down by whether they have a college degree or not.

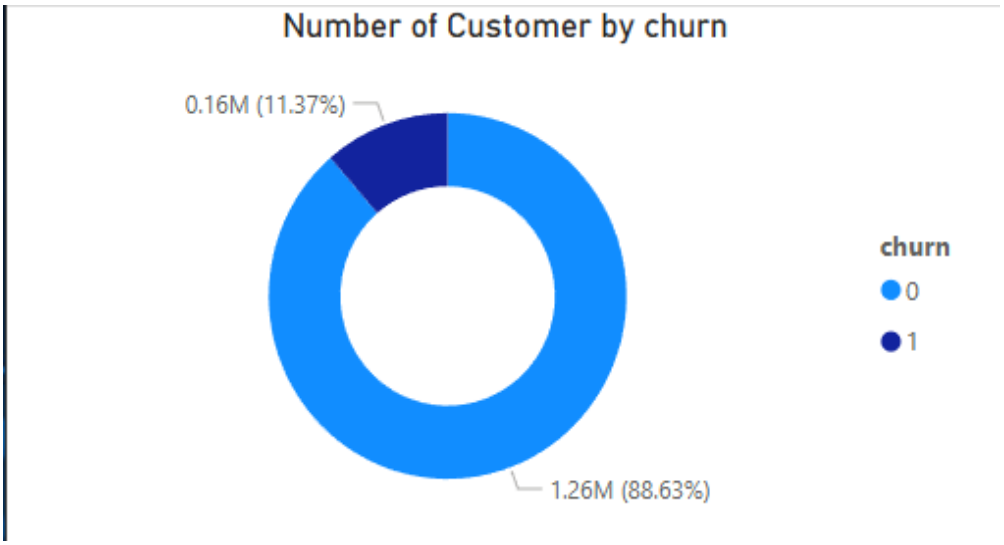
The output tells us that for both with or without a college degree, the number of married customers that churned is more than the single ones. A reasonable explanation to this is that married people tend to have way more commitment than single people. They have their own family to feed, therefore need a better insurance package to cover more people. To avoid more churns from happening, it is suggested that this company improve their insurance plan for the married, like giving discounts for every family package offered.

DATA VISUALISATION

DASHBOARD



VISUALIZATION 1



Analysis of Data:

The donut chart above shows how customers in the insurance industry divide between those who switched and stayed with their provider. The chart shows two groups of customers: those who stayed (labelled 0) and those who left (labelled 1). The chart shows that 1.26 million

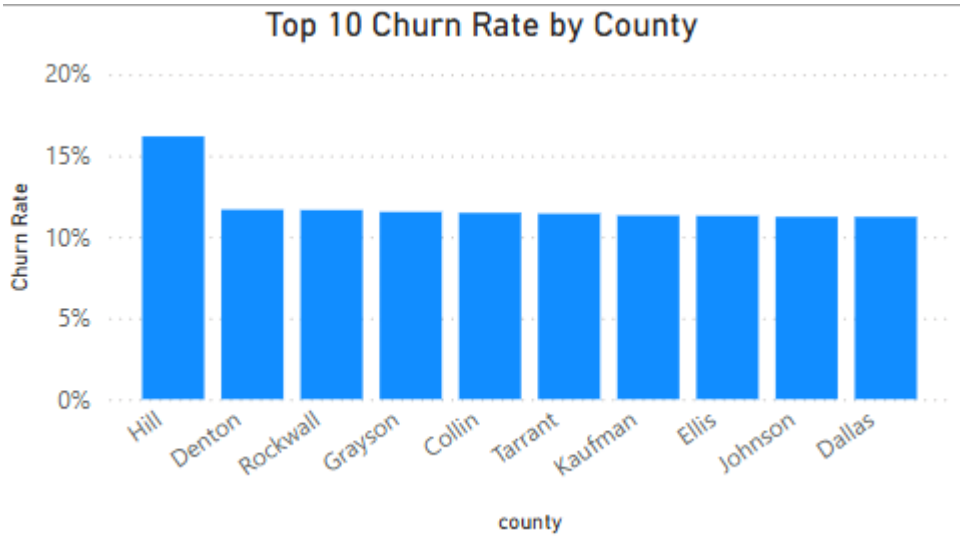
customers (88.63%) stayed loyal to their policies while 0.16 million customers (11.37%) ended their coverage during the observed period. Most customers stay with their providers but a significant number still leave.

Results:
The visualization demonstrates that the customer turnover rate remains at 11.37% which indicates satisfactory customer retention. The substantial size of the customer base results in the loss of around 160,000 customers which requires further analysis of their profiles and reasons for disengagement.

Discussion:
Insurance providers face higher expenses in winning new customers than they do in maintaining their current clientele thus an 11% churn rate represents a significant issue. The research underscores that studying factors which include age alongside marital status and credit score and geographic region helps understand customer departure patterns. The research project seeks to detect emerging patterns which lead to identifying vulnerable groups of customers.

The understanding of these insights helps develop retention strategies which include customized promotional deals for different customer groups and individual engagement plans and better service approaches. The project's main objective of building a sustainable customer-centric insurance business benefits from this approach. The visualization creates a base structure which directs subsequent examinations of detailed attributes found in the dataset.

Visualization 2



Analysis of Data:

The bar graph displays the top ten counties which experience the greatest number of insurance customers who leave their current provider. Each county shows its churn rate using percentage values. The county with the highest churn rate is Hill County which exceeds 16% and becomes followed by Denton and Rockwall which both maintain rates above 11.37% (as shown in Visualization 1). The chart demonstrates which geographic regions experience greater insurance customer loss because these areas show higher attrition rates.

Results:

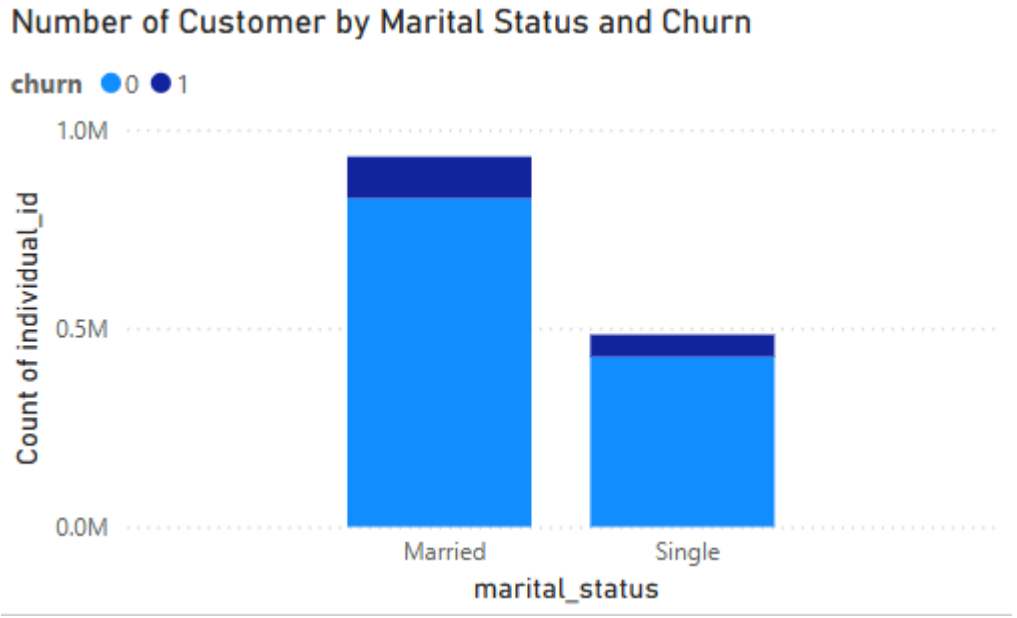
Hill County ranks highest among all locations for customer churn which indicates localized problems with either service coverage or pricing or stronger competition in the area. The counties of Denton together with Rockwall and Grayson maintain comparatively high churn rates. In comparison to the highest churn counties Dallas and Johnson maintain moderate levels of churn which provides opportunities for improvement.

Discussion:

Location-specific interventions can be developed using the geographic patterns of customer churn. The highest churn rate in Hill County suggests that focused research should identify the main factors including customer demographics and socioeconomic aspects and competitor activity. The presence of younger or lower-income customers in Hill County could create a connection between demographic characteristics and reduced policy loyalty. The strategic

distribution of resources becomes more effective when organizations analyze these patterns to decide between marketing focus or enhanced local service or price modifications. The analysis allows insurance providers to identify and target high-risk areas for customer retention initiatives which helps achieve business sustainability through improved efficiency and reduced attrition..

Visualization 3



Analysis of Data:

A bar chart displays customer quantity distribution according to marital status (Married vs Single) together with their churn activity. The dark blue sections represent churned customers (churn=1), while the light blue sections represent retained customers (churn=0). The data shows that married people represent most customers but experience significant customer losses.

Results:

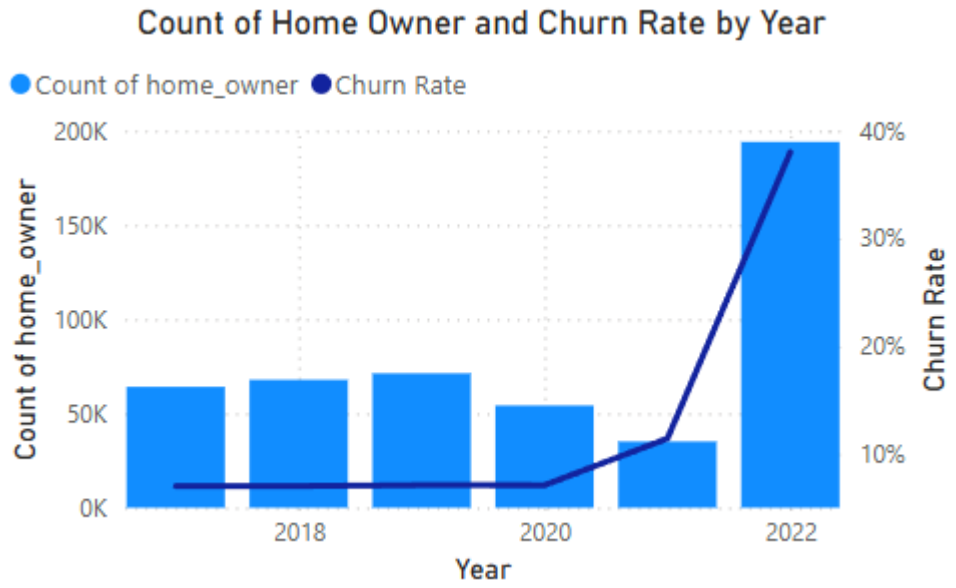
Married customers exist in larger numbers yet their absolute number of churned customers exceeds the number of single customer churn. Single customers might demonstrate a marginally greater churn percentage in comparison to their entire customer base. Both customer segments lose customers to attrition but their departure likely stems from different factors.

Discussion:

Marital status assessment in relation to churn offers valuable knowledge about customer life events that affect insurance selections through family obligations and monetary stress. Customers who remain single typically face less provider loyalty because they lack ties which makes it easier for them to change providers but married customers tend to focus more on policy value and family coverage needs.

The discovery validates specific retention methods that include customized family packages for married clients along with adaptable insurance options for single individuals. It supports the research objective of identifying demographic-specific customer attrition risks to establish individualized retention approaches.

Visualization 4



Analysis of Data:

A dual-axis chart shows two different metrics across the years 2017–2022: the count of home owners (bar graph) and the churn rate (line graph). From 2017 to 2021 the number of homeowners stayed mostly stable but experienced a minor decline in 2021 followed by a rebound in 2022. The churn rate stayed low from 2017 through 2021 but experienced a sharp increase in 2022 to reach 35%.

Results:

In 2022 the churn rate experie2022 witnessed a significant surge in churn rates because economic disturbances as well as market transformations and competitive dynamics impacted homeowners more than other groups. The observation shows that being an owner does not protect individuals from losing customers while external elements like inflation and policy adjustments and recessionary pressures seem to have substantial influence on retention patterns. Time-based evaluation of customer retention models becomes essential because this visualization demonstrates the importance of monitoring yearly performance metrics. The increased churn rate demonstrates why companies must adopt flexible engagement strategies

during external pressure periods which directly supports the project's objective of scientific and enduring customer retention methods.nced a significant surge although the total number of homeowners only slightly increased. The data indicates that churn rates moved independently from the total customer base while external elements or specific conditions caused higher customer attrition during that year.

Discussion:

The dramatic rise in churn rate in 2022 could be attributed to economic disruptions, market changes, or competitive shifts that disproportionately affected homeowners. It indicates that ownership status alone does not shield against attrition, and that temporal factors (e.g., inflation, policy changes, or a recession) might play a critical role.

This visualization supports time-based analysis in customer retention modeling and emphasizes the need for year-over-year performance tracking. The surge in churn rate underlines the importance of agility in customer engagement strategy, especially during periods of external stress aligning directly with the project goal of evidence-based and sustainable customer retention.

6.0 CONCLUSION

In conclusion, this project has applied data warehouse concepts to organise, process, and analyse customer churn data in a structured architecture. The data source from the CSV file that was loaded into the PostgreSQL environment undergoes ETL (Extract, Transform, Load) process. This allowed the data to be properly analysed using two techniques, Online Analytical Process (OLAP) operations via pgAdmin and data visualisation via Power BI. Hence, with this application we can generate insights that would not be visible in raw data alone. The architecture and approach has turned complex data into actionable business knowledge.

The study sets out to understand what really drives customer churn in the insurance industry. The findings paint a clear picture in answering several objectives. First of all, personal, financial, and geographical factors each play a big role, with younger customers, especially single ones, having higher rates on leaving the insurance provider. Those with lower incomes and home owners are more likely to churn as well. Though, the rate of customer churn among homeowners also increased significantly. For geographical factors, Hill county records the highest churn rate out of any other counties. Moreover, many of the individuals come from cities like Dallas and Fort Worth, suggesting there may be deeper factors about location that might affect.

Beyond the identification of who's leaving, the analysis has uncovered hidden patterns that explained why it happened. It became clear that it is no coincidence that the churn happened. It follows recognizable insights tied to age, income, home ownership, and geographical location. This allows insurance companies to shift from guessing to knowing, as well as help design targeted strategies to reduce churn before it happens in future.

Most importantly, the findings of the project help open the door for action. By understanding which type of customers or demographics are most likely to churn and why it happened, companies can set up and arrange their strategies, by offering more relevant communication, support, or value to the group of people who need it the most. This way, insurance firms have the opportunity to act earlier and potentially build stronger, long-lasting relationships with their customers.

7.0 REFERENCES

Auto Insurance churn analysis dataset. (2023, April 30). Kaggle. https://www.kaggle.com/datasets/merishnasuwal/auto-insurance-churn-analysis-dataset?select=demographic.csv

Kagan, J. (2020, June 23). *What is an insurance score? Definition, purpose, and example*. Investopedia. https://www.investopedia.com/terms/i/insurancescore.asp

PRUCare Family. (n.d.). Prudential Malaysia. https://www.prudential.com.my/en/products-health-insurance/critical-illness-plans/prucare-family/

What is a good credit score? | Experian. (n.d.). https://www.experian.co.uk/consumer/guides/good-credit-score.html